The backtest system

2026-11-01. Replays the desk's own entry rules over the archived minute data on the VPS, priced on real option prices, and stores the trades in the same shape as live ones so the Trade Analysis page reads them with no new screens.

This document is the whole program: what was added, what each file does, how the pieces serve each other on the VPS, what the engine assumes about the data, and — the part that matters most — what has been proven and what has not.

Read this first. The engine is written and covered by 185 tests, but **it has never run against the real archive.** Every test runs against a stand-in that answers the same queries. A passing test proves the wiring, not the data. Nothing it produces should be quoted until the two items in section 10 are done.

---

1. What changed, in one table

AREA	CHANGE
New Python package	`app/scripts/backtest/` — the replay engine, deploying to `/opt/backtest`
New systemd service	`backtest-worker` — long-running, picks queued runs off MongoDB
New dashboard screen	**Backtest** on the rail: pick legs and dates, Run, past runs
New API route	`/api/backtest` — list, queue, open, delete
Trade Analysis	now reads a backtest run behind a "Backtest, not live" banner
`lib/strategy/types.ts`	new exit reason `out-of-band`
`lib/strategy/analysis.ts`	its label, "Ran past the archive"
New Mongo collections	`backtest_runs`, `backtest_trades`, `backtest_meta`

Nothing in `registry.ts` moved. The live rules, their versions and their parameters are untouched — the engine mirrors them and refuses to start if the mirror has drifted.

---

2. A run names setups, not strategies

The long rule is two legs joined by an "OR" in its own spec:

- **A** — price is below the major resistance and crosses it, pulls back to within 0.4% of the 21 EMA, then turns up 0.2% off that low. **Long, buys a call.**
- **B** — price is holding above the major support, crosses another marked level, and is above the 21 EMA. **Long, buys a call.**
- **C** — price breaks the major support, rallies back to within 0.4% of the 21 EMA, then rolls over 0.2% off that high. **Short, buys a put.**

C was already a strategy of its own. A and B were bundled, which meant there was no way to find out which of the two was carrying the result. A run now names **setups**, so each leg can be replayed on its own:

```
run.py now --from 2026-09-01 --to 2026-10-31 --setups A
run.py now --from 2026-09-01 --to 2026-10-31 --setups B
run.py now --from 2026-09-01 --to 2026-10-31 --setups C
run.py now --from 2026-09-01 --to 2026-10-31 --setups A B C
```

This is a **reporting split, not a rule change.** Each leg still runs under its own strategy's parameters and version; nothing about how a trade is taken has changed.

The separate runs do not add up to the combined one

This is the one number that will mislead if it is not understood. Two reasons:

1. **The three-position cap is desk-wide.** A leg replayed on its own gets all three slots. Run together, the legs compete for them.
2. **One trade per stock per day, per strategy.** If B takes a stock in the morning, A cannot take that same stock later — and it does not even produce a signal, because the rule checks the cap before emitting one.

A test demonstrates exactly this: one stock, one price path, that offers B at 09:20 and A at 09:26. Run A alone and you get the 09:26 trade on the 1010 strike. Run B alone and you get the 09:20 trade on the 1020 strike. Run both and you get **only B** — a different answer from either, and from their sum.

So: use the single-leg runs to see which leg is worth keeping. Use the combined run to see what the desk would actually have made.

---

3. How one session is replayed

The shape mirrors the live desk's day.

STEP	WHAT HAPPENS	SOURCE
1	Fibonacci pivots from the previous session's high, low and official close	`cm_bhavcopy`
2	Confirmed by that session's per-strike call/put OI — nearest expiry, ATM±8, 1.8× skew, within 0.8%	`fo_bhavcopy`
3	Major resistance and support = least-gap levels clearing 4× and 5×	—
4	21 EMA seeded from the last 84 settled 5-minute bars of previous sessions	`spot_1min`
5	The session runs forward **one minute at a time**	`spot_1min`
6	Open positions are managed **before** new signals are considered	—
7	On a signal: `strike_offset = 0` at that minute gives the strike to buy	`option_1min`
8	The book gate decides whether the contract is worth touching	§6
9	The position is followed by its **strike**, minute by minute, to its exit	`option_1min`

Step 6 is not cosmetic. The cap is shared across every running leg, and a signal at 11:04 can only be taken if something has closed by 11:04. Running each position to completion the moment it opens would decide that cap against a book that does not exist yet.

---

4. Every file, and what it is for

The engine — `app/scripts/backtest/`

FILE	PURPOSE	ON THE VPS
`backtest/strategies.py`	The registry **mirrored** from `registry.ts`, plus the setup index. `check_against_registry()` re-parses the TypeScript and reports drift.	The worker refuses to start on drift
`backtest/levels.py`	Fibonacci pivots, the OI confluence, the major picks. Ported from `lib/eod`.	Pure computation
`backtest/signals.py`	The three entry state machines and the EMA. Ported from `lib/strategy/signals.ts` and `ema.ts`.	Pure computation
`backtest/position.py`	Sizing, the trailing ladder, exits. Ported from `position.ts`.	Pure computation
`backtest/fills.py`	Advances one position by one minute; the out-of-band rule.	Pure computation
`backtest/gate.py`	The book gate, as much as the archive can answer, with the `clear` / `approx` / `rejected` mark.	Pure computation
`backtest/db.py`	Every read from the PostgreSQL archive. Paise→rupees and UTC→IST happen here and nowhere else.	Reads the archive
`backtest/runner.py`	One run: sessions, the minute loop, the summary, the run id.	The work
`backtest/results.py`	Writes runs and trades to MongoDB; publishes the archive coverage.	Writes results
`backtest/worker.py`	Polls MongoDB for queued runs and executes them one at a time.	**The service**
`backtest/config.py`	`.env`, folders. Reads secrets, prints none.	Config
`settings.py`	The gate floors and run limits. **Not** the strategy rules.	Config
`run.py`	The command line: `check`, `coverage`, `levels`, `session`, `now`, `queue`.	How you drive it
`deploy/install.sh`	One-time setup: venv, `.env`, systemd. Safe to re-run.	Install
`deploy/backtest-worker.service.in`	The systemd unit template.	Install
`requirements.txt`	`psycopg[binary]`, `pymongo`.	Install
`README.md`	The package's own documentation.	—

The tests — `app/scripts/backtest/tests/`

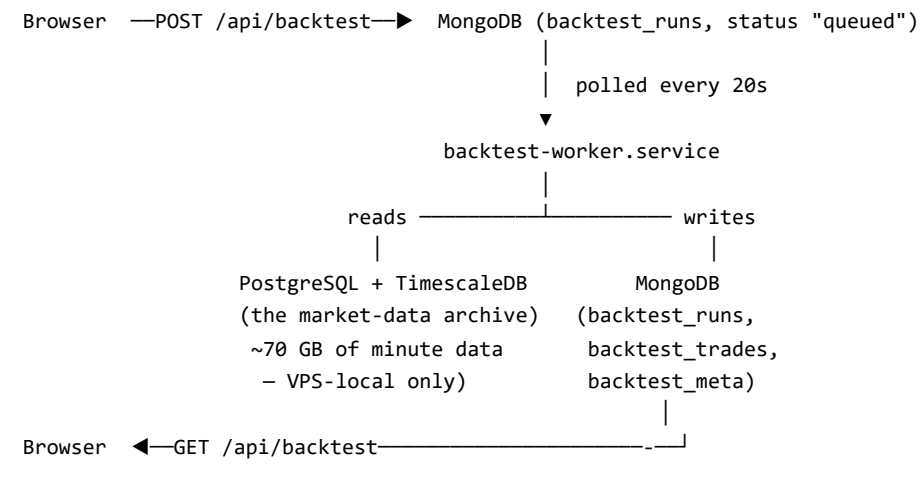
FILE	WHAT IT COVERS
`test_core.py`	The ladder against its documented numbers, the EMA, the levels, all three setups firing and — more important — **not** firing
`test_fills.py`	Stops, gaps through the stop, the flatten, and the edge of the archive
`test_session.py`	One whole session end to end, the gate refusing, run identity
`test_strategies.py`	A, B and C apart and together; the short buys a put; the desk-wide cap
`fake_archive.py`	The stand-in archive: answers exactly the queries `db.py` asks, for any number of stocks

185 checks, no database and no network. They exist because a silent error here does not crash — it produces a plausible P&L that is wrong, which is the worst failure a backtest has. The suite was checked against deliberate sabotage: making the short buy a call fails 2 checks, making the cap per-leg instead of desk-wide fails 5.

The dashboard — `app/src/`

FILE	PURPOSE
`components/Backtest.tsx`	The panel: setup checkboxes, dates, Run, past runs, per-leg breakdown
`lib/backtest/store.ts`	Reading runs and trades from Mongo; queueing one
`lib/backtest/setups.ts`	The setup catalogue, derived from `registry.ts` so it cannot drift
`lib/backtest/key.ts`	The run id — the other half of `runner.py:run_key`
`app/api/backtest/route.ts`	`GET` list / one run · `POST` queue · `DELETE` remove
`components/TradeAnalysis.tsx`	Now takes an optional `runId`; shows the banner
`components/SectorRail.tsx`, `Desk.tsx`	The rail entry and the wiring

5. How the pieces serve each other on the VPS



The dashboard never replays anything. It writes a queued row and returns. The worker is the only process with the archive in reach, and a twenty-minute replay has no business inside a web request.

The worker is a separate service from the downloader on purpose: a three-month backtest must not delay the nightly download, and a download failure must not stop a backtest. It runs at `Nice=15` so it always yields to the dashboard and the download.

The archive coverage is published by the worker. The dashboard cannot query PostgreSQL, so it cannot know what dates exist. The worker writes a snapshot into `backtest_meta` on start and hourly, and the screen shows it beside the date pickers. Without it a range can be picked that the archive does not cover, and the run comes back empty with nothing on screen to say why.

The run id, and why it is what it is

A run's `_id` is `sha1("A,B,C|2026-09-01|2026-10-31")` truncated to 16 characters — the legs and the dates, nothing else. Computed identically by `runner.py:run_key` and `lib/backtest/key.ts`.

It deliberately leaves out the gate floors and how much data existed, because **the dashboard cannot see PostgreSQL** and must be able to compute the id to answer "you have already run this" the moment the dates are picked. What it leaves out is

recorded *on* the run instead — `gateFloors`, `dataRevision`, `conditions`, the rule versions — and shown beside the result.

If those two functions ever disagree, nothing errors: dedup silently stops working and every repeat replays from scratch. `tests/test_session.py` pins the Python side to the literal hash so that cannot happen unnoticed.

---

6. The book gate — what it reproduces and what it guesses

The live gate in `tradeable.ts` exists because of a measured failure: 9 of the first 36 trades died inside ten seconds and between them accounted for the entire net loss. Every one had been filled far above where the contract was trading — SHREECEM at 470.95 against a last print of 299.70, on a strike carrying 125 contracts of open interest against 11,625 at the fattest strike. Nothing had moved. There was no book.

A backtest without the gate reproduces the desk *before* the gate existed, and will report losses the live rule would never have taken.

#	LIVE CHECK	IN THE ARCHIVE?
1	Book is two-sided	**No** — no quotes stored
2	Ask within 2% of a fresh print	**No**
3	Spread under 2% of mid	**No**
4	Spread under 30% of the stop distance	**No**
5	The contract traded at all	**Yes, exactly** — `option_1min.volume`
6	Strike OI ≥ 3% of its chain's heaviest	**Yes, exactly** — `fo_bhavcopy.open_interest`

Checks 5 and 6 are the same test on the same numbers. Checks 1–4 are all spread tests, and a spread is the one thing the archive does not hold — they are stood in for by liquidity, on the gate's own reasoning that the books which failed it were thin ones.

Every trade carries a mark:

MARK	MEANING
`clear`	comfortably above the liquidity floors — the live gate would almost certainly have allowed it
`approx`	passes, but near them — the live gate might have refused it. Taken, counted, and marked
`rejected`	below them, or failing check 5 or 6 — not traded, as the desk would not have

Each run reports its headline **with** and **without** the `approx` trades, and the banner on the Trade Analysis page shows both. If those two numbers are far apart, the result rests on the approximation rather than on the rule.

The floors are not yet calibrated. `settings.py` has `GATE_CALIBRATED = False`. Step 0 sets them against the 155 trades the live desk actually took — those all passed the real gate, so a floor that rejects many of them is too strict.

---

7. What the data does, and the three traps

`strike_offset` is a label, not a contract

The downloader asks Dhan for seven *series* per stock, `ATM-3` … `ATM+3`, and **Dhan decides what "ATM" means minute by minute** from where spot is. As the stock moves, the real strike behind the `ATM` series changes.

Reading a held position with `WHERE strike_offset = 0` returns a series that jumps from one contract to another every time spot crosses a strike boundary. **Nothing about the result looks broken** — every number is plausible and all of them are wrong. The offset is read once, at the entry minute, to pick the strike; after that the position is followed by `strike_paise`.

A contract that leaves the band closes the trade

Only ATM±3 was downloaded each minute, so a position that runs far enough simply stops being in the data — no row, no error. That silence is told apart from "did not trade this minute" by reading the band out of the data itself: the lowest and highest strike actually present for that symbol, expiry and minute. Inside it, carry the last price and hold. Outside it, close at the last known price with the exit reason **`out-of-band`**.

Those are real closed trades, not errors — but they were ended by the edge of the archive rather than by the rule. A run with many of them needs reading with that in mind, which is why they are countable under their own name.

Prices are unadjusted

`cm_bhavcopy.close_paise` is NSE's official close, unadjusted. The pivots are built from the *previous* session and the EMA is seeded from previous sessions, so on the morning after a split yesterday's prices are on the old scale and today's on the new one — every level would land far from the market. `corporate_actions.adjust_factor` is applied over the window being read.

Two more worth knowing

- **Expiry is two different questions.** The levels scan takes the nearest expiry **strictly after** the session (`chain.ts` skips `expiry <= session`, or a scan on expiry evening reads a chain that has already settled). The contract a trade **buys** takes the nearest expiry **on or after** the trading day, so the one expiring that day still counts.
- **The option universe changes every month.** `liquidity_rank` picks the most liquid F&O stocks per month on the 20 trading days before it. Spot and the NSE files cover all ~200 stocks; options only the ~100 selected. Read from `liquidity_rank`, never from a fixed list.

---

8. Stored results

`backtest_runs` — one row per run, keyed by the run id: the legs, the dates, status and progress, the summary, `summaryExApprox`, `gateCounts`, `bySetup` (each leg's own figures), the conditions it ran under, and any error.

`backtest_trades` — the same shape as a live `strategy_trades` document, plus `runId` and `gate`. Keeping the shape identical is the whole point: it is what lets the Trade Analysis page read a backtest with no new screens — the pivot filter, the "how far it ran" panel, the trailing what-if, the equity curve, all of it.

`backtest_meta` — one document, the archive coverage snapshot the screen shows.

---

9. Operating it on the VPS

Install, or re-install after a deploy

```
cd ~/<the MO1 repo on the VPS>
git pull
sudo bash app/scripts/backtest/deploy/install.sh
```

Creates `/opt/backtest` with a venv and `logs/`, copies `DATABASE_URL` across from the downloader's own `.env` where it can, installs and enables `backtest-worker.service`. Safe to run again.

Then fill in the Mongo connection string and restart:

```
sudo nano /opt/backtest/.env
sudo systemctl restart backtest-worker
```

Check it over

```
cd ~/<the MO1 repo>/app/scripts/backtest
/opt/backtest/venv/bin/python run.py check
```

Reports whether the strategy mirror still matches `registry.ts`, and whether both databases answer.

What the archive actually covers — run this first

```
/opt/backtest/venv/bin/python run.py coverage
```

Prints what `option_1min` holds and over what dates, complete stock-days per month, the spot and NSE file ranges, the monthly option universe, and any outstanding gaps. **Nothing else is worth doing until this has been read** — it decides which date ranges exist at all.

Working up from one session

```
/opt/backtest/venv/bin/python run.py levels  --day 2026-10-15
/opt/backtest/venv/bin/python run.py levels  --day 2026-10-15 --symbol RELIANCE --verbose
/opt/backtest/venv/bin/python run.py session --day 2026-10-15
/opt/backtest/venv/bin/python run.py session --day 2026-10-15 --setups A
```

`levels` and `session` print and store nothing, so they are free to run as often as you like. Check the levels against what `daily_levels` already holds for that session before trusting anything downstream.

A range

```
/opt/backtest/venv/bin/python run.py now   --from 2026-09-01 --to 2026-10-31
/opt/backtest/venv/bin/python run.py now   --from 2026-09-01 --to 2026-10-31 --setups A
/opt/backtest/venv/bin/python run.py queue --from 2026-09-01 --to 2026-10-31
```

`now` replays in the terminal and stores the result. `queue` hands it to the worker instead, which is what the dashboard button does. A range already run comes back instantly with its stored result; `--force` replays it anyway.

The service

```
sudo systemctl status backtest-worker
sudo journalctl -u backtest-worker -f
sudo systemctl restart backtest-worker
tail -f /opt/backtest/logs/worker.log
```

The tests, on the VPS

```
cd ~/<the MO1 repo>/app/scripts/backtest
/opt/backtest/venv/bin/python tests/test_core.py
/opt/backtest/venv/bin/python tests/test_fills.py
/opt/backtest/venv/bin/python tests/test_session.py
/opt/backtest/venv/bin/python tests/test_strategies.py
```

---

10. What has not been done

1. The engine has never run on real data. Every test runs against a stand-in. The first real run will find things no test can: whether the archive holds the stocks and dates assumed, whether the levels match `daily_levels`, whether the option series are complete enough to price a fill.

2. The gate floors are invented. `GATE_CALIBRATED = False`. They need setting against the 155 live trades, which all passed the real gate.

3. The validation run. Replay 21 Sep – 31 Oct — the period the desk has already traded — and compare trade for trade against `strategy_trades`. They will not match exactly; the gate approximation alone guarantees that. But every difference should have a reason that can be named. **If they cannot be reconciled, the engine is wrong and no later result means anything.**

Limits that will not go away

- **The fill price is the entry minute's close**, not an ask. There are no quotes in the archive. This flatters every trade by roughly half a spread and is the single largest reason a backtest result will read better than the live record.
- **The rule is evaluated once a minute**, where the live engine sees every tick.
- **A minute that both stops the trade and clears a rung counts as a stop** — the conservative reading, and the only one a 1-minute bar allows.
- **`option_1min` has no open interest and no bid/ask.** Minute OI was dropped deliberately to save disk; daily OI comes from `fo_bhavcopy`.

---

11. One behaviour worth knowing

A signal refused because the book was full **does not come back** when a slot frees. This is the live behaviour, not a shortcut: `engine.ts` calls `evaluateSymbol` first — which marks the setup `spent` the moment it fires — and only then asks the book for a slot. The setup is already spent by the time the refusal happens, and the stock has to set itself up again from scratch.

On a busy morning, the desk does not queue the signals it could not take. Worth knowing before reading a run where `book full` appears often.